Lab 5: Time Synchronization and Consensus

18-449/649 Carnegie Mellon University

Release: 11/09/2026 Due: 11/18/2026, 11:59pm Version 1.0

What you are building

This lab focuses on the software elements and introduces protocols to provide consistency: consistency in terms of time synchronization and consistency in terms of network state.

At a high level, your nodes will all run Raft which will store in its logs the current network state (i.e., which links and nodes are up). The nodes will use this network state to perform routing.

The raft leader will also act as the time synchronization leader, periodically sending out time sync messages for the other nodes to use to synchronize time and to calculate local drift. We will implement simple time synchronizations by introducing a reference message: i.e., one node, the time master, sends it every 20 ms, and every other node computes a global time from it. Given this global time, the blinkers, so the front and rear will be aligned.

Fault tolerance: When a node goes down, the others elect a leader, the leader takes over the reference CAN message, the vehicle enters a failure mode with hazards flashing in step.

When a CAN link goes down instead, nothing is elected because nodes should still be reachable, i.e., via relaying messages.

Part 1: Node Setup and Raft Clients

We will add a Raft agent (which has all the raft logic), a log file for the agent to use, a config file also for the agent to use, and a Raft client.

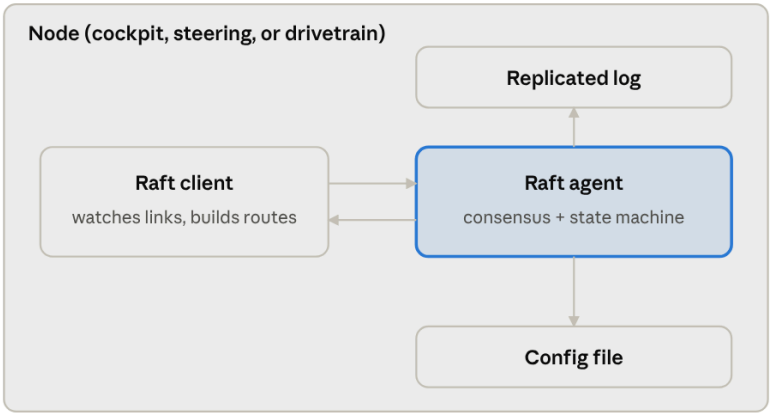


Figure 1 · processes running on each node

The client sends messages to the Raft agent to report link and node failures or additions, and to request the current network state. A Raft client should send its messages to the leader. If it sends a message to a follower instead, the follower redirects the client to the leader.

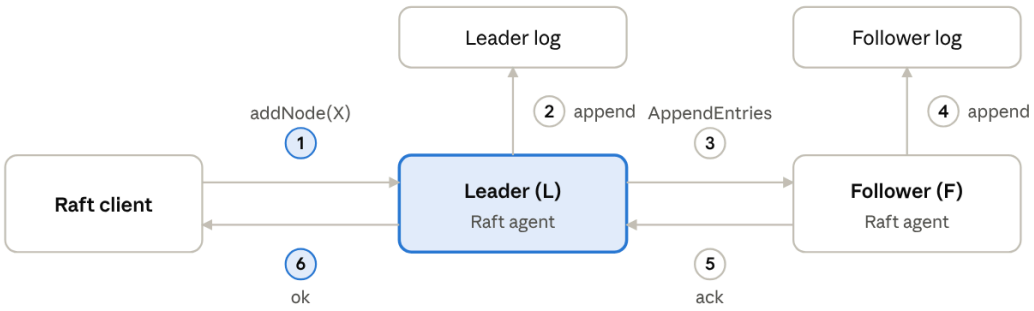


Figure 2 · a client write, from request to acknowledgement

Part 2: Raft Leader Election

Each node participates in the Raft cluster and runs a Raft agent. You must support the Raft optimization in which nodes start with a pre-defined leader, so no election is held at boot.

The configuration file must be identical on every node. It should include the following keys:

Key	Meaning	Default
N	Size of the cluster. A majority is N/2 + 1.	3
default_leader	Node ID of the pre-defined leader, or none. When a node ID is given, every node boots in term 1 with that node as leader and no election is held.	cockpit
heartbeat_ms	How often the leader sends its heartbeat. The heartbeat is also the clock-synchronization message (Part 6), so this is also the sync period.	20
missed_heartbeats	How many consecutive leader heartbeats a follower may miss before it declares the leader dead and starts an election.	3
election_timeout_jitter_ms	Random range added to the election timeout so that two followers rarely time out together. A follower's timeout is missed_heartbeats × heartbeat_ms + U(0, election_timeout_jitter_ms).	39

log_size	Number of events the Raft log holds	30000
nodes	The ID of each node, so the state machine can name it in log entries.	a unique ID for each node
Debug	Boolean value, if set, then the raft node prints out election msg, append msg,	Yes

Part 3: The Replicated Log

Each log entry has three fields: (1) a term, (2) an index, and (3) an event. The event types are described below. Recall that each Raft agent includes a state machine that applies the events in the log to build a global network state.

Entry type	The leader appends it when	Effect
COCKPIT_OK, COCKPIT_ERROR	Cockpit heartbeats stop or resume	Enter or leave failure mode
NODE_X_UP, NODE_X_DOWN	Node X comes up or goes down	Mark node X up or down
CANLINK_X_Y_UP, CANLINK_X_Y_DOWN	The CAN link for node X on bus Y comes up or goes down	Mark the link up or down

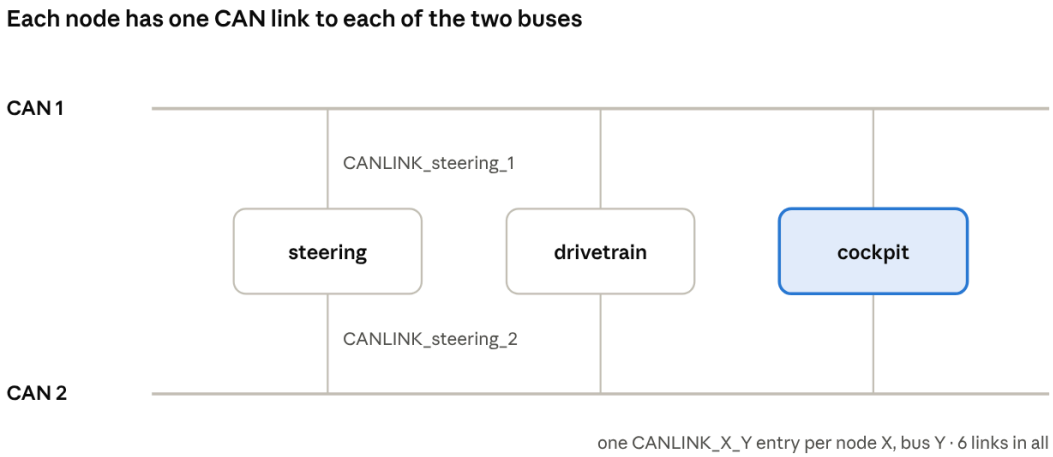


Figure 3 · the CAN links that the log tracks

Figure 3 shows the topology the log describes: three nodes, each attached to both CAN 1 and CAN 2. Every CANLINK_X_Y_UP or CANLINK_X_Y_DOWN entry names one of these six links, so the state machine can mark exactly which link changed.

Part 4: Raft State Machine

The leader creates the appropriate log events based on the client updates it receives. The state machine applies those events and supports the following functions:

Each function either alters the graph state or retrieves it. The four mutators are applied as the state machine replays log entries; getNetwork is the only read.

Function	Kind	Arguments	What it does	Applied on
addLink(X, Y)	Alters graph	Node ID X, bus Y	Marks the link between node X and CAN bus Y as up, adding it if it is not yet in the graph.	CANLINK_X_Y_UP
dropLink(X, Y)	Alters graph	Node ID X, bus Y	Marks the link between node X and CAN bus Y as down. The node and bus remain in the graph.	CANLINK_X_Y_DOWN
addNode(X)	Alters graph	Node ID X	Adds node X to the graph (or marks it alive again) with no links until addLink is applied for it.	NODE_X_UP
dropNode(X)	Alters graph	Node ID X	Removes node X from the graph along with every link attached to it.	NODE_X_DOWN
getNetwork()	Retrieves graph	None	returns the current network graph captured by the state machine.	Called by the Raft client when it builds its routing table (Part 5.1)

Part 5: Rethinking System Architecture and Dynamics

Using Raft to maintain network state, together with time synchronization, changes two key behaviors: how routing is done, and how relaying is done.

5.1 Routing with network state

Each node builds its routing table from the log held by its local Raft agent. To support this, the state machine provides getNetwork(), which returns the network state the state machine has built from the log. getNetwork() returns the entire network graph; the node then distills that graph into the relevant subset it needs for routing.

5.2 Relaying messages between nodes

Failing two links creates an edge case. The first link failure is detected correctly by the client on the leader and recorded in the leader's state. When the second link fails, however, it may take several rounds, depending on which node is the leader, for the information to propagate and for the routing tables on all Raft agents to be set up correctly.

Part 6: Time Synchronization

Time synchronization consists of two tasks: coordinating clocks across nodes, and accounting for local drift between synchronization events.

To keep all nodes synchronized, the leader includes its clock in the heartbeat messages it sends to the followers. Followers use this clock-synchronization message both to recalibrate their local clocks and to measure drift.

6.1 Clock synchronization

When a follower receives a clock value from the leader, it must account for network delay. Unlike PTP/NTP, where the client requests the time and uses the request/response round trip to estimate latency, TTCAN uses the properties of the CAN bus itself to estimate network latency.

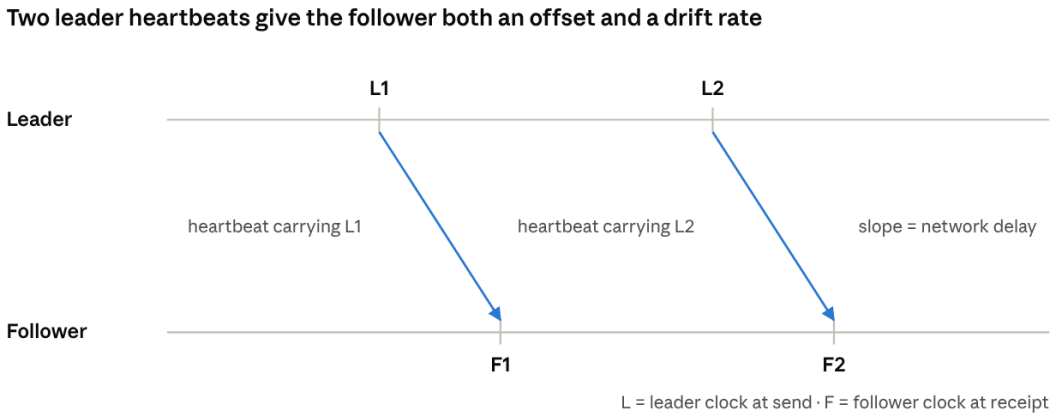


Figure 4 · clock synchronization from two consecutive heartbeats

Figure 4 shows two consecutive heartbeats. The leader stamps each with its clock (L1, L2); the follower records its own clock on receipt (F1, F2). The first heartbeat sets the follower's clock; the pair gives the relative drift.

$$\textbf{follower_clock} = L_1 + \textbf{network_delay}$$

Updating Zephyr on Followers: You should update the system time on each follower (e.g., using clock_settime()) based on the adjusted leader time.

6.2 Accounting for drift between synchronizations

Between synchronization events, local clocks drift continuously. Followers must estimate this drift and correct for it. Use the leader's periodic heartbeats to measure the drift rate and apply the correction locally.

$$\textbf{relative_drift} = \textbf{leader drift / follower drift} = (L_2 - L_1) / (F_2 - F_1)$$

Accounting for drift: Between updates from the leader, you should use a drift ratio to reduce the impact of local drift on the clock

6.2 A new global_gettime() function

Add a `global_gettime()` function that returns the current global time, applying your drift adjustment to the local clock between synchronization events.

Also add a periodic task that calls `global_gettime()` and prints the drift error it is currently correcting for, so the correction can be observed during the demo.

6.4 A PPS output

Each node must drive a pulse-per-second (PPS) output on a GPIO pin: one rising edge at every whole second of `global_gettime()`. Because every node derives the edge from the same global time, the edges from all nodes should line up on a scope; the spread between them is a direct measure of how well your synchronization is working.

On each PPS edge, the node should also print a line to its console containing:

- the wall-clock time (`global_gettime()`) at the edge, and the local clock reading at the same instant;
- the current drift estimate (the drift ratio from 6.2, and the correction applied since the last heartbeat);

These printouts are what you will use in the report to quantify drift and jitter, and the scope trace of the PPS pins is what the TA will look at during the demo.

Part 7: the Demo

The demo must show three things: that leaders work correctly, that the logs stay intact, and that time synchronization performs as expected. Run the three scenarios below in order.

Scenario	Startup mode	Steps	What to examine
1. Link failure, default mode	Default mode	1. Start the devices. 2. Introduce a link failure.	Network state after the failure
2. Link failure and recovery	Leader setup mode	1. Start the devices. 2. Introduce a link failure. 3. Recover from the link failure.	Network state after the failure, and again after recovery
3. Leader failure	Leader setup mode	1. Start the devices. 2. Fail the leader.	Network state after the failure; the blinkers, to confirm time synchronization survives the election

For each scenario, record the network state as returned by getNetwork() on every node and confirm that all logs agree.

Extra credit: a cloud node with Redis and a GUI (15 points)

The vehicle's log is the source of truth; the cloud node is a mirror that lets someone watch the vehicle from a browser. The Pi is the only node with a network interface, so it is the uplink. A VM in the cloud joins your raft cluster as a client.

Setup. One small cloud VM (a free-tier instance on AWS, GCP or Azure is enough) running Redis and a small web app (e..g Flask or FastAPI with websockets, Node with Socket.IO)

What the Pi publishes.

Redis structure	Key	Written when	Holds
Stream (`XADD`)	`vehicle:log`	every committed entry	term, index, type, payload, global time
Hash with TTL	`vehicle:heartbeat`	every 1 s	Pi wall-clock, global time, FL/RL offset as the Pi last measured it

Publishing is asynchronous and best-effort: a queue on the Pi drained by its own thread, never on the CAN receive path, never delaying `REF`, `APPEND` or `CTRL`; show that in a scope capture. If the uplink is down the queue keeps the last 1,000 entries and resumes from the last acknowledged stream id.

What the GUI shows. The three nodes with alive/dead and bus A/B status, leader and time master marked; the count of events the leader is holding.

Demonstrate. Run the Part 5 checkpoint with the GUI open. The TA should see the cockpit go dead, the STM32 leader take over, the held-event count rise, and the replayed events arrive marked as replayed when the Pi returns. Because the Pi *is* the cockpit, the GUI goes stale during the outage and catches up afterwards; explain in one paragraph what it would take to keep it alive through a cockpit failure and why the Nucleos cannot do it.

Item	Points
Redis and GUI deployed securely (no open Redis, HTTPS with login)	3
Log, events, state and heartbeat published as specified, with resume after uplink loss	5
GUI shows all five views and the staleness banner	4
Demonstration and the one-paragraph explanation	3

Checkoff

Run the following sequence for a TA with your system in front of them. Steps 1–3 are the Lab 3 checkoff compressed; the rest are new.

1. **Cold start, `LEADER=cockpit`.** Power up with the proxy stopped: all three nodes in the error state, hazards in step (we scope FL, RL and HZ), LEADER on the Pi only, SYNC on all three. Start the proxy; the system leaves the error state.
2. **Bus traffic.** `candump -c any` shows every catalog ID including `REF` from the Pi every 20 ms and empty `APPEND` every 100 ms, and no `REQ_VOTE`. We will read the cycle counter and the master time in `REF`.
3. **Blinkers.** Left, right, cancel; FL and RL within 1 ms over ten edges.
4. **Link faults, no election.** Pull any one node's A jumper, then Bfs. DEG LEDs as in Lab 3, term unchanged on every node's console, one `MEMBER` entry per change at the same index everywhere. We pick the node.
5. **The hop with the Pi as master.** Pull N1 off B and N3 off A. Wheel still steers, blinks and brakes; FL and RL still within 1 ms; term unchanged; two `MEMBER` entries committed; an `EVENT` from N2 reaches N3's log via the Pi.
6. **Pi down.** Power off the Pi. Within 100 ms: error state, motor braked. Within 1 s: one STM32 LEADER on, `REF` from it, hazards at 4 Hz on FL and RL in step with no step larger than 100 µs at the handover.
7. **Pi back.** Power on the Pi. It rejoins as a follower, the held events appear on its display exactly once in order and marked replayed, then live events; hazards return to 2 Hz; LEADER stays on the STM32.
8. **Random leader.** Reboot everything with `LEADER=random`. An STM32 is leader. Power off that STM32: a new leader within 1 s, `REF` moves, no failure mode. Power off the Pi: failure mode, held events, replay on return, with whichever node is leader. Then the seed that makes the Pi win, and step 6 again.
9. **Relayed sync with an STM32 master.** With an STM32 leading, pull the scenario (c) jumpers so the other STM32 is cut off. SYNC stays on at the cut-off node, REF_RX still toggles there, FL and RL within 1 ms. Restore: no step on the edges.
10. **Isolation and test button.** As in Lab 3, plus: the term changes only if the isolated node was leader, and exactly once.
    .

Expect verbal questions throughout, directed at everyone on the team. The guiding questions are a fair preview; so is the question of what would change with four nodes.

Deliverables

The grade is split between the system working in front of us and the written analysis that predicted it would.

Demo	Points
Live demo – the checkoff sequence above	40
Checkoff questions, answered by the team	20

Report (to be submitted to Gradescope)	Points
Github link	4
Discussion of time drift adjustment: detecting when to use time drift and how it is applied. Describe (1 page max)	13
Description of Raft state machine (1 page max)	13
Description of raft client functionality and how it impacts routing over CAN especially during multi-link failures (1 page max).	10